

Experience Teaching Software Project Management in both Industrial and Academic Settings

Philippe Kruchten
University of British Columbia
Vancouver, Canada
pbk@ece.ubc.ca

Abstract

This paper relates seven years of experience teaching Software Project Management both in academia as part of an undergraduate software engineering program and to software engineering graduate students, and to practitioners in industry. It explains some of the difficulties and constraints for such a course. It describes the current syllabus and its rationale. The course is constructed based on a conceptual model of software development that accommodates a wide range of process models, traditional and agile, large and small. The course is illustrated by drawing from a range of concrete processes: RUP®, DSDM®, MSF®, Scrum and XP, of software engineering standards (from IEEE and ISO) and a few project management tools. The paper then maps this course to the IEEE SWEBOK (Software Engineering Body Of Knowledge), to IEEE Standard 1490, better known as the PMBOK (Project Management Body of Knowledge), and more particularly to the recent IEEE-CS/ACM SE2004 (Software Engineering Curriculum 2004), showing how the course can be made an integral part of a well-rounded software engineering program.

1. History and context

After spending 30+ years in industry in various roles: designer, researcher, architect, manager, process author, the author joined academia, in the Electrical and Computer Engineering (ECE) department of the University of British Columbia (UBC). He had taught some aspects of project management to industrial practitioners, mostly in relation with the deployment of the Rational Unified Process® [1, 2], but the academic context was very different. Most university students have usually very little clue as to what a project means, they see any software project as mostly lots of late night programming, and debugging heroics, and have no experience dealing with more than 3 or 4 co-workers, mostly co-opted.

This paper summarizes lessons learned over the last six years in building and delivering a course on “Software Project Management”, 8 times in academia, and 4 times in industry, not counting deliveries by some licensees, to gather some of the “reusable” bits and pieces for the readers faced with similar challenges.

2. Motivation and Constraints

In the Rational Unified Process® (RUP®), we had defined Software Project Management as “the art of balancing competing objectives, managing risks, and overcoming constraints to successfully deliver a product which meets the needs of both customers (the payers of bills) and the users” [2].

2.1. Common Misconceptions

There are several misconceptions about software project management that this course must address in some ways, and they provide some insights into the overall project management ‘philosophy’ of the course.

- *The Project Manager (PM) is the boss.* She decides who does what, when, and where and make sure that her directives are followed. On the opposite to this command-and-control, almost military view, we see the PM as a facilitator that removes any obstacle she can in front of the advance of her team, and cleans up some of the mess left behind.
- *Microsoft Project® is all you need to manage software projects.* There is even in my city a course on software project management entirely organized around N various ways of running this tool. In reality, this tool is not very useful for software projects, because of the lack of clear work breakdown structure. A CM tool and an issue tracking tool are far more useful than Gantt charts and CPMs.
- *A PMP® certification is more useful than a little knowledge in software design and coding.* While the PMBOK [3] is a valuable catalogue of skills, it has a heavy up-front planning emphasis that does not marry itself well with software projects, as they require lot of invention and creativity, and need adaptation and flexibility. Moreover the PM who does not understand what the team is doing will be sitting in an ivory tower until doomsday arrives.
- *We should be able to manage software projects like any other projects.* Well, no. And it is not by lack of trying over the last 30 years, but the software development community has come to realize that some of the key tenets of “classic” project management, such as a sequential waterfall lifecycle, a knowledge of predefined work-breakdown structures, accurate effort estimation techniques, precise metrics of progress, are all missing in software for various reasons described elsewhere [4].

Software project management is not merely a project management sub-process coexisting peacefully with many technical sub-processes: it is deeply entangled with requirements, architectural design, testing, release management, *etc.* J. McDonald [5] and W. Royce [6] have also described what makes software project management difficult, and what implications it has on teaching it.

2.2. Learning Objectives

What are the learning outcomes for such a course? From the SE2004 [7] (p.15), we want the student to be able to “reconcile conflicting project objectives, finding acceptable compromises within limitations of cost, time, knowledge, and existing systems and organizations.” We want students to be able at the outset to be able to distinguish various types of software projects, to select or adapt a lifecycle adapted to the project at hand, to estimate efforts and cost, to track progress on various fronts: time, risks, deliveries, *etc.*, assess quality, both of the product and the process, understand the key issues in managing people and teams, and be able to reflect on their own projects to improve over time their management practice. They also need to understand ethical and legal issues, as well as some of the difficulties communication and coordination related to distance or cultural difference in global software projects, as these are rapidly becoming dominant model. These objectives are also aligned with what practitioners like Tockey think we should prepare students for [7].

2.3. Other Constraints for a Course on Software Project Management

Here are some of the key constraints to the problem, besides the lack of personal experience the university students can rely on:

- *Timing*: typical courses in North American academia are 3 hours a week for 13 weeks. (I like to teach day-long courses, and I had a hard time at first distilling this material in 1 hour ‘chunks’!). On the industrial front, it is hard today to “sell” courses longer than 3 days.
- *Compliance to SE2004 [8] and SWEBOOK [9]*: the course must integrate with the software engineering curriculum developed jointly by ACM and IEEE, based in part on IEEE’s Software Engineering Body of Knowledge.
- *Alignment or differentiation with the PMBOK [3]*: though the IEEE has adopted in bulk the *Guide to the Project Management Body of Knowledge* as IEEE standard 1490:1998 [10], the PMBOK has much more of a civil engineering or construction project flavor and is often maladapted to software projects.
- To these natural constraints I added a totally arbitrary one: trying to “reuse” large amount of the course or at least the course material to teach in industry, through Kruchten Engineering Services Ltd. and two training companies: *Software Education* in New Zealand (softed.com) and *Software Productivity Center* in Canada (spc.ca) that is, make the course modular enough to be adapted to a wide range of different context.

3. Course Syllabus & its Rationale

The course is organized in 13 sections (see Table 1). Within most of the 11 core sections, the structure is similar: what are we trying to solve: key concepts/definition, traditional approaches, issues with them in software development, new approaches (e.g., agile), applicable standards, examples of techniques and tools; some examples are shown in table 1.

1. Introduction

This section introduces a conceptual framework for software project management (see section 4), and the most common frameworks used in industry: PMBOK, CoBIT [11], ITIL [12], Prince 2. It presents an overview of the industrial processes that will be used throughout the course: RUP, MSF [13] and DSDM [14], agile methods such as eXtreme programming [15] and SCRUM [16, 17]. It finally brings in the picture industry standards, and the various standardization bodies: IEEE, ISO, RTCA, etc. The case study that will run throughout the course is presented.

2. Managing risks and uncertainty

If the objective of the project manager is to drive the project to success, then she should be constantly aware and actively fighting anything that can come in the way of success: risk. Tom Gilb said it very eloquently [18]. Starting the course with risks allows to introduce early the key concept of iterative development, and to explain that many choices made on the way are not always “optimal” or ideal, but are driven by risks mitigation, or the path of lesser risk. Note that the PMBOK, while not excluding iterative lifecycle, has still a strong waterfall flavour, more amenable to construction and administrative projects, where lots of detailed planning can or must happen up-front. “Plan the work, then work the plan.” It is important to introduce risks and uncertainties early, as they explain many of the activities or practices introduced later.

3. Managing time

The two questions that the software project manager has to answer again and again from the start to the end are:

- “When will you be done?” and
- “How much is it going to cost?”

As software development is very “people intensive,” cost is strongly linked to effort: person-months. Besides, time cannot be altered, stopped or rewound back, so the flow of time is a great source of anxiety for software project managers, and even more so for large projects with rather artificial intermediate milestones. So this focus on time and time flow gives a second opportunity to motivate iterative development, from a different perspective, and it allows early tackling of the crucial and still very immature issue of estimation of effort and cost. After looking at the big picture: overall effort and schedule, we examine more tactical time management: iteration planning, backlog management. Finally this leads to the more conventional topics of planning and scheduling (critical path, etc.).

4. Managing Quality

The 3rd question software projects managers have to be ready to answer is:

- “...and will the quality be sufficient for us to meet our return on investment objectives?”

That is: to find buyers, to get the final payment, etc... and to maybe be allowed to develop version 2, or start another product development. And this is where software project management meets testing (though quality is far broader than what testing can tell us), and where we introduce first the concept of measurement, validation and verification, etc.

5. Managing objective and scope

This section is where we connect project management with the requirements activities, the managing of requirements throughout the project, the control of requirement ‘creep’, and the assessment of actual progress, with technique ranging from earned-value to burn-p charts.. We bring more measurements to assist in monitoring progress. (Bridge with a specification course)

6. Managing complexity

For large projects, linking with software architecture and how architectural design can support the project manager in organizing teams and structuring the timeline; socio-technical congruence. (Bridge with an architecture course)

7. Managing changes

We introduce here software configuration and change management, the standards, practices, tools and methods, as well as the concept of a change control board. These offer the PM other opportunities of measurements. We also look into the process that leads to a customer release.

8. Managing software assets

We look at software as an asset: issues of reuse (development with or for reuse), of components. We look into open source development, which leads to copyrights, software licenses, software patents, etc., and other intellectual property issues.

9. Managing people

Yes, finally, project managers do also manage people, too! And this section is long to come (need to keep some suspense...). This has two part: one focussing on the team, with organizational structures, communication, collaboration, notions of shared mental model and trust, etc., a second part more focus on the individual with a glimpse in the the HR cycle: hiring, training, coaching, compensating, rewarding, promoting, and even firing people We bring in issues such as professionalism, ethics, and cultural issues, especially with global,

distributed software projects. To quote SE2004 [8] (p.44): “Ethical, legal, and economic concerns, and the notion of what it means to be a professional, should be raised frequently.”

10. Managing external stakeholders

Large projects use contractors (or you are the contractor), or they off-shore or outsource part of the development; we examine how to manage these external stakeholder, including the customer or its representatives. We also look at progressive acquisition: how large procurement office use iteration to acquire process. And we introduce concept of software development governance.

11. Managing products

Looking at the intersection between product management and project management; what needs to be done to have a complete product: delivery, deployment, support, etc., and the more complicated issues of software that is embedded in other products.

12. Managing the software process itself

This section looks into the software development itself, and focuses on how to improving the quality of the product by looking at the process. It covers practices on to select, adapt, deploy, and improve the software engineering process itself. It makes the compulsory detour via the CMMI.

13. Conclusion

We can now look at a whole project over its whole lifecycle. And maybe introduce some more advanced topics: safety-critical systems, impact of new constraints such as the US Sarbanes-Oxley regulation [19], etc., and more complex governance [20].

While it is not possible to manage a project in class, we introduce simulation games to illustrate concepts such as iteration planning, velocity (developed with James King at SoftEd), or one on risks and technical debt developed with the SEI (see the two posters in the same CSEE&T proceedings), and one case study. We produced little videos to illustrate some techniques. For example a 3-minute skit on the “Daily Stand-up meeting from hell” was developed with SoftEd in Wellington, NZ. There is no textbook exactly matching this course, but we provide in academia a mapping to the textbook by Hughes and Cotterell [21].

4. The Conceptual Models behind the Course: the Frog and the Octopus

A course on software project management can easily turn into a combination of

- very abstract, high-level principles and frameworks (PMBOK, CMM [22], Prince 2, RUP, ...)
- and a boring collection of recipes, with little means to put them into practice, unless you can play with a tool (e.g., Microsoft Project, or COCOMO).

To counterbalance this we have developed 2 models:

1. a *conceptual model of software development*, which captures key concepts that exist in some for or another in any software engineering project (the Frog)
2. a *variability model* which describes factors that vary across the whole spectrum of possible software engineering projects (the Octopus).

The two models are presented right at the beginning of the course and they are used systematically to:

- establish some common ground terminology
- describe many practices, techniques: what they do, how they work, etc. by expressing them in terms of the model

- analyze their value, usefulness, or applicability in various contexts
- find what is common or different across various processes, frameworks, standards
- perform analysis of case studies: why did this work, or not?
- and more generally develop critical and reflective thinking about software project management, rather than learn cute tricks or enter a funny sect.

The two parts of the model are more fully described elsewhere [23, 24], but the following 2 subsections will give you the gist of it. Note that this is not a very general “theory of software engineering”, it is only intended to support reasoning about project management.

4.1. Conceptual Model of Software Development

The four cornerstone concepts are:

1. **Intent:** what is it that we want ultimately to accomplish. Intent may take various form (SRS, use case, backlog, test suite, defect list, etc.)
2. **Product:** the ultimate software that is the outcome of the project
3. **Work:** all the activities and tasks that have to be accomplished to realize the intent into product. A process is a kind of template for element of the work.
4. **People:** software development is still essentially a human- and knowledge-intensive activity

These four key concepts each have 3 key attributes:

5. **Time:** intent and product evolve over time, especially with iterative development; work and even people will vary over time
6. **Risk and Uncertainty:** software development project are most often plagued with lots of unknowns and risks, more so than other types of projects
7. **Quality:** quality of the product, which is influenced by the quality of the work, the people and intent.

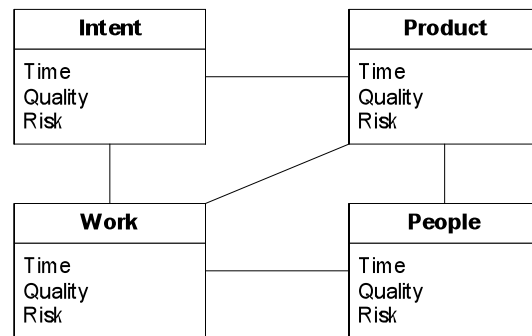


Fig.1 - Concepts

Finally, 8. **Value** is associated with intent and product: we need to assign expected value to the intent to guide development of the product over time, while the 9. **Cost** of the development is associated with the work and the people (a characteristic of software development not shared with other engineering disciplines, such as civil engineering).

Altogether a software development 10. **Project** is all the **work** that **people** have to accomplished over **time** to realize in a **product** some specific **intent**, at some level of **quality**, delivering value to the business at a given **cost** [24].

4.2. Variability Model of Software Development

Beyond very general concerns such as the business domain (i.e., type of industry), the culture of the country and of the organization, there are 8 key factors that have a big impact on the management of a software project [23]. They are, in roughly a decreasing order of importance:

1. **Size:** how big is the system to develop (in SLOC, function points, of person-months)
2. **Criticality:** how many people die if the system fail, or how many billions of euros are lost
3. **Age of the system:** greenfield development, brownfield, evolution of legacy system, maintenance
4. **Business model:** how is the project remunerated for its effort; in-house development, commercial product, software embedded in another product or system, open source development, research...
5. **Team distribution:** colocated, geographical distributed (outsourcing, etc.)
6. **Volatility** of the environment: how stable are the requirements and the surrounding business environment
7. **Stable architecture:** how much of a stable architecture exists at the start of the project
8. **Governance:** what are the external rules imposed to the project to control its trajectory and how formal are they.

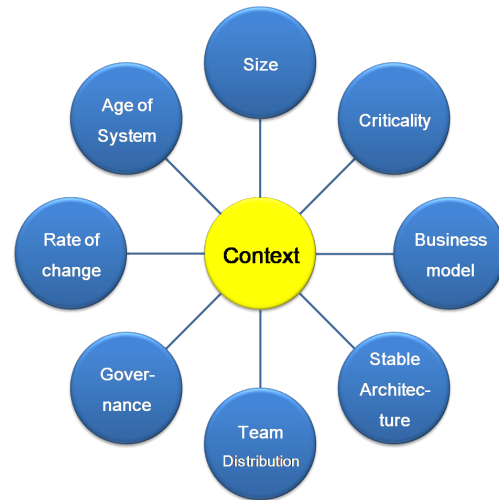


Fig.2 - Variables

The reader will now recognize the rationale for the outline of the course (section 3), which explores in the most appropriate sequence the various concepts: risk & uncertainty, time, quality, value, cost, intent, product, work, people, project, ... trying to minimize forward references, and allowing some repetition and revisitation for refinement and re-enforcement (learning). Figure 2 and 3 become leitmotifs in the course.

5. Lessons learned

The main lesson was the importance of developing, then introducing right at the start, the conceptual model (cf. section 4). Both in academia and in industry, this allows to position the exposition of the classroom material, but also frame the discussions, and the assignments or examinations at the proper level, avoiding in particular sterile debates about agility (or not), by putting reflection and discussions in the proper context.

High modularity of the lecture material (currently in 22 sections) allows to adjust to different formats for delivery, and to rapidly tailor a course for a specific customer.

Alternating between industry and academia has enriched the course; on one hand keeping it grounded in reality; on another hand developing more synthetic models and explanations.

It took several iterations to find the proper sequence of topics. The resulting sequence is not too far from that of Tatnal's [25]

• Lessons learned *in industry*

About 21 to 24 hours of face-to-face time imposes some cuts in the material. But we can also draw from the participants experience.

For copyright reasons, it is often not possible to make available most standards, and most technical papers (reading material).

When delivered in-house, we can perform some deeper tailoring of the course based on the concrete process used, or the issues they want to resolve by such training.

- **Lessons learned in academia**

We noticed a great difference in comprehension between students who have been part of a co-op program, or who are returning to school after working in industry, relative to those whose only project and team work experience is limited to a triplet of buddies in 3rd year. Partly for the reason above, we are still unsure as to whether the Software Project Management course should precede or follow the Software Engineering capstone project course.

6. Outside of the classroom

Section 3 covered mostly the classroom (or face-to-face) part of the course and this is pretty much all of it when the course is delivered in an industrial setting, which consists of 2 to 4 days of continuing education. This covers the ‘knowledge’ part in Bloom’s taxonomy [26]. In a university setting, there are other related activities, to go to the ‘comprehension’ and ‘application’ levels: Assignments, Readings, Evaluations, Projects.

A) Assignments

The students are given a description of a software project: context, initial requirements, “vision” in the RUP sense [27], and they build in steps a complete Software Development Plan, in the sense of IEEE 1058:1998, or using a RUP template, and a Business Case.

B) Software Engineering Project

The best way for student to really ‘understand’ the concepts is to put them in practice; the ‘application’ level in Bloom’s taxonomy. Assignments are too narrow and somewhat contrived, they let us dig in a technique, but not become a project manager. From SE2004 [8] p.57, again: “A capstone project course is essential in a software engineering degree program. [...] It gives students the opportunity to undertake a significant software engineering project, in which they can deepen their knowledge in many of the [software engineering curriculum] areas.” Umphress and his colleagues [28] found that software capstone projects should “cultivate a process culture, that agile processes provide a bridge from ad hoc programming assignments to organized project work.” At UBC, software engineering student in their last year complete the Software Engineering Capstone Project, a 5-credit course (so equivalent in weight to 65 hours of classroom, or 200 hours of individual work for each student). To avoid reducing this to a mere massive effort in late night programming, students are forced to work in groups of 7 or 8, on a project challenging enough to avoid the “one programmer and seven looking over her shoulder” effect. For example, in 2006 they implemented a tool to allow co-editing a document through the web (similar to Groove’s editing function).

7. Coverage of SE2004

SE2004 [8] decomposes the knowledge into 10 knowledge areas (KA), and each KA into small knowledge units (KU). This course covers (cf. Table 1, column labeled ‘SEEK KA/KU’):

- All units of the MGT (Management) and PRO (Software process) areas.
- Most of QUA (Quality), all units of the Professionalism part of PRF (Professional Practice), and some of VAV (Verification and Validation).
- And a few units from EVO (Evolution).

Table 1: Sections of the course with topics, relative size, standards, tools practices, & coverage of the SE2004 [8]

#	Topic	%	Standards	Practices	Tools	SEEK KA/KU
1	Introduction <i>PM frameworks, process framework, “management 101”</i>	10	PMBOK (IEEE 1490) IEEE 1058, DSDM, RUP, MSF, CMMI, Agile Processes, ISO & IEEE std	Plan-do-check-act Software development plan	PM’s dashboard	MGT.con.1-2 PRO.con.1 PRO.imp.1-2 PRO.imp.2
2	Managing Risk	10	IEEE 1540	Iterative development		MGT.pp.6 PRO.imp.2-3
3	Managing Time <i>Estimation, scheduling</i>	18	ISO 14143, ISO 19761	Function points and other “points”: UCP, FFP	COCOMO etc., SLIM, Msft Project, Various “planning games”	MGT.pp.1-5 PRO.imp.2-3
4	Managing Quality	15	ISO 9126, IEEE 730, IEEE 1061, etc. ISO 9000-3	GQM, Reviews, pair programming, TDD	Bug tracking tools	QUA.cc QUA.std. pro, pca VAV.fnd, par MAA.rv.1
5	Managing Objective and Scope	7		Daily stand up, Backlog	Earned value systems, Burn up/down charts, ReqPro	MGT.pp.1 MGT.ctl.2
6	Managing Complexity <i>Relation with architecture and system engineering</i>	3	IEEE 1471	Metaphor, 4+1 views, aspects		DES.ar PRF.psy.4
7	Managing Changes <i>Internal and external changes</i>	10	IEEE 838 & 1042	CCB, “triage”	SCM tools (e.g., subversion, ClearCase), tracking tools	MGT.cm.1-7 MGT.ctl.1 MAA.rfd.5
8	Managing Assets <i>Reuse, open source, etc</i>	4			Repositories	CMP.ct.2 PRF.pr.3
9	Managing Products	2				FND.ec
10	Managing People	10	SWE code of ethics and Prof. practice	Org. charts; Sustainable pace; Performance appraisals		MGT.per PRF.pr
11	Managing External Stakeholders	5	ISO 12207, IEEE 1062	Customer on site		
12	Managing Software Process	3	ISO 15504, IEEE1074	Retrospective		PRO.con.3-4-5 PRO.imp.4-5-6
13	Wrap-up	3			Dashboard revisited	

Acknowledgments

Many thanks to Shane Hastie and James King (SoftEd), Hans van Vliet, Patricia Lago (VU), Joe Marasco (ex-Rational) for their inspiration, help and feedback.

References

- [1] IBM, "Rational Unified Process," 2003 ed. Cupertino, CA: IBM Rational Software, 2003.
- [2] P. Kruchten, *The Rational Unified Process--An Introduction*, 3 ed. Boston, MA: Addison-Wesley, 2004.
- [3] PMI, *A Guide to the Project Management Body of Knowledge: Pmbok Guide*, 4 ed. Washington: Project Management Institute, 2008.
- [4] P. Kruchten, "Putting the "Engineering" into "Software Engineering"," presented at the Australian Software Engineering Conference (ASWEC 2004), Melbourne, Australia, 2004, IEEE Computer Society.
- [5] J. McDonald, "Why Is Software Project Management Difficult? And What That Implies for Teaching Software Project Management," *Computer Science Education*, vol. 11(1), pp. 55-71, 2001.
- [6] W. Royce, *Software Project Management: A Unified Framework*. Reading, MA: Addison-Wesley, 1998.
- [7] S. Tockey, "Recommended Skills and Knowledge for Software Engineers," presented at CSEE&T 1999, New Orleans, LA, USA, 1999, IEEE Comp Soc.
- [8] ACM/IEEE, "Computing Curriculum - Software Engineering (v.3.1)," ACM/IEEE February 6th, 2004 2004.
- [9] P. Bourque and R. Dupuis, Eds., *Guide to the Software Engineering Body of Knowledge*. Los Alamitos, CA: IEEE CS Press, 2001.
- [10] IEEE, "IEEE Std 1490-1998: IEEE Guide Adoption of PMI Standard, A Guide to the Project Management Body of Knowledge," 1998.
- [11] IT Governance Institute, *CobiT 4.0 Control Objectives for Information and related Technology--Control Objectives, Management Guidelines, Maturity Models*. Rolling Meadows, IL: IT Governance Institute, 2005.
- [12] OGC., *Information Technology Infrastructure Library (ITIL) v3*. London: UK Office of Government Commerce (OGC), 2007.
- [13] Microsoft., "Microsoft Solutions Framework v3.0 Overview," Microsoft, Redmond, WA June 2003 2003.
- [14] J. Stapleton, *DSDM, Dynamic Systems Development Method*. Reading, MA: Addison-Wesley, 1998.
- [15] K. Beck, *Extreme Programming Explained: Embrace Change*. Boston: Addison-Wesley, 2000.
- [16] M. Cohn, *Succeeding with Agile: Software Development Using Scrum*. Upper Saddle River NJ: Addison-Wesley Professional, 2009.
- [17] K. Schwaber and M. Beedle, *Agile Software Development with SCRUM*. Prentice-Hall, 2002.
- [18] T. Gilb, *Principles of Software Engineering Management*. Reading, MA: Addison-Wesley, 1988.
- [19] US Government, "US Public Law 107 - 204 - Sarbanes-Oxley Act of 2002," US Government Printing Office, Washington 2002.
- [20] S. Chulani, C. Williams, and A. Yaeli, "Software Development Governance and Its Concerns," presented at Software Governance workshop at ICSE 2008, Leipzig, Germany, 2008, ACM.
- [21] B. Hughes and M. Cotterell, *Software Project Management*: McGraw Hill Higher Education, 2005.
- [22] M. C. Paulk, B. Curtis, M. B. Chrissis, and C. V. Weber, "Capability Maturity Model for Software, Version 1.1.," Software Engineering Institute, Pittsburgh, PA SEI-93-TR-024, 1993.
- [23] P. Kruchten, "Contextualizing Agile Software Development," presented at EuroSPI 2010, Grenoble, France, 2010. Available at <http://pkruchten.wordpress.com/process/>
- [24] P. Kruchten, "A conceptual model of software development," in *Contribution to the SEMAT working group at semat.org*, 2009. Available at <http://pkruchten.wordpress.com/process/>
- [25] A. Tatnall and G. Reyes, "Teaching IT Project Management to Postgraduate Business Students: A Practical Approach," *Journal of Information Technology Education*, vol. 4 pp. 153-166, 2005.
- [26] B. S. Bloom, M. Englehart, E. Furst, W. Hill, and D. Krathwohl, Eds., *Taxonomy of educational objectives: The classification of educational goals: Handbook I, cognitive domain*. New York: Longmans, Green, 1956.
- [27] P. N. Robillard and P. Kruchten, *Software Engineering Process with the UPEDU*. Boston: Addison-Wesley, 2003.
- [28] D. A. Umphress, T. D. Hendrix, and J. H. Cross, "Software process in the classroom: the Capstone project experience," *IEEE Software*, vol. 19(5), pp. 78-81, 2002.